Chandubhai S Patel Institute of Technology

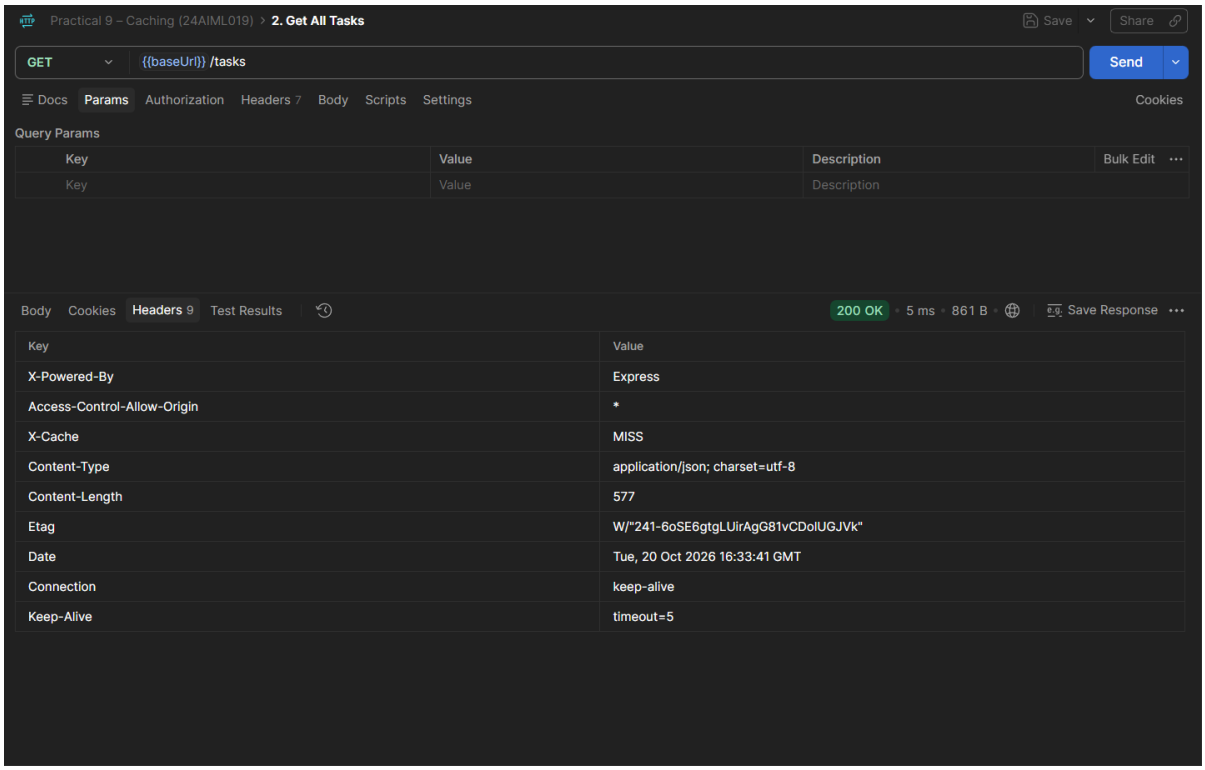
CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY
FACULTY OF TECHNOLOGY AND ENGINEERING
B. TECH (IT/CE/CSE/AIML)

DEPSTAR

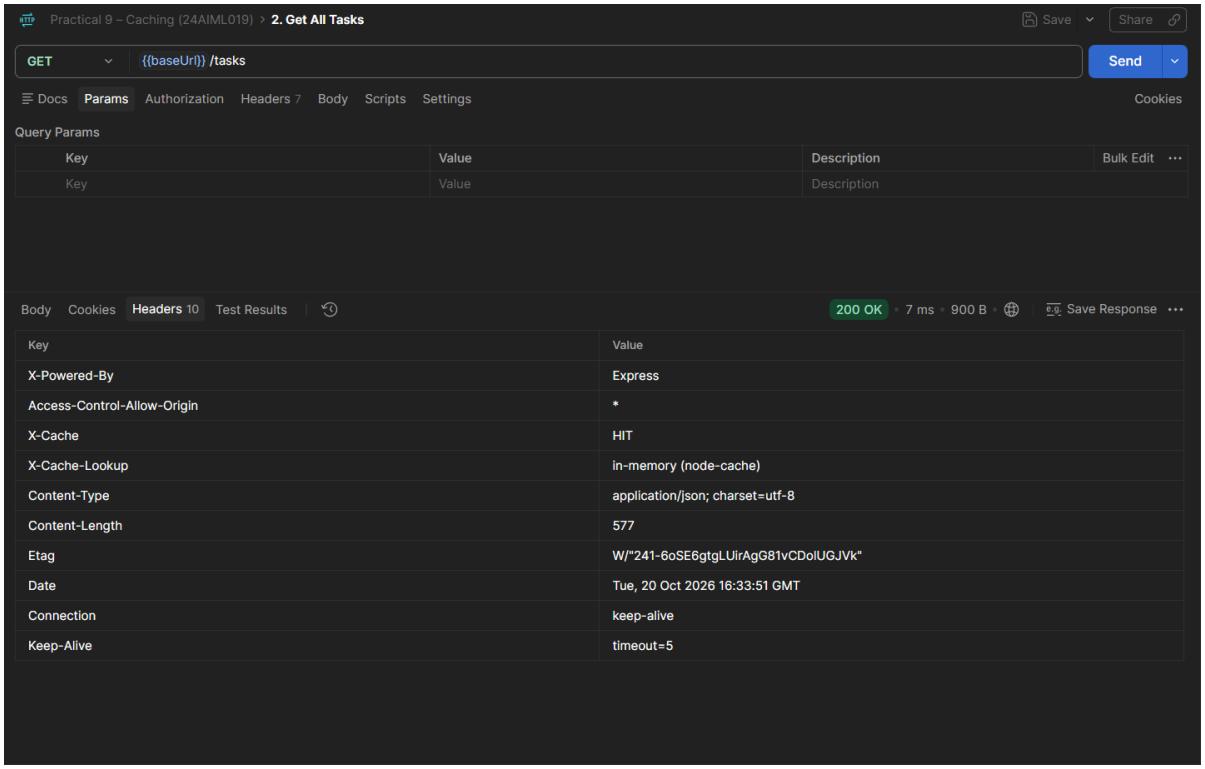
Subject: Advanced Web Development Frameworks (ITUE301) | Semester: 5th

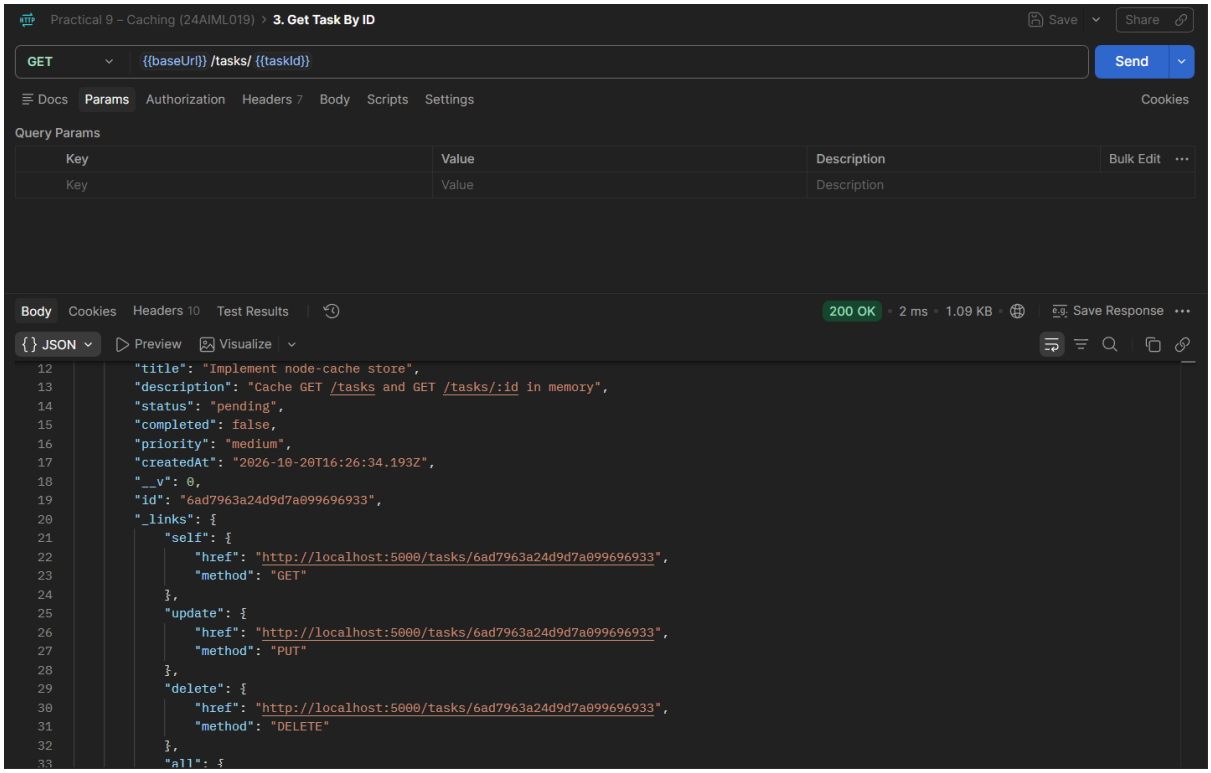
Practical Statement: To implement server-side caching and measure its impact on API response time.

Screenshots (Supplementary Problems are also implemented):

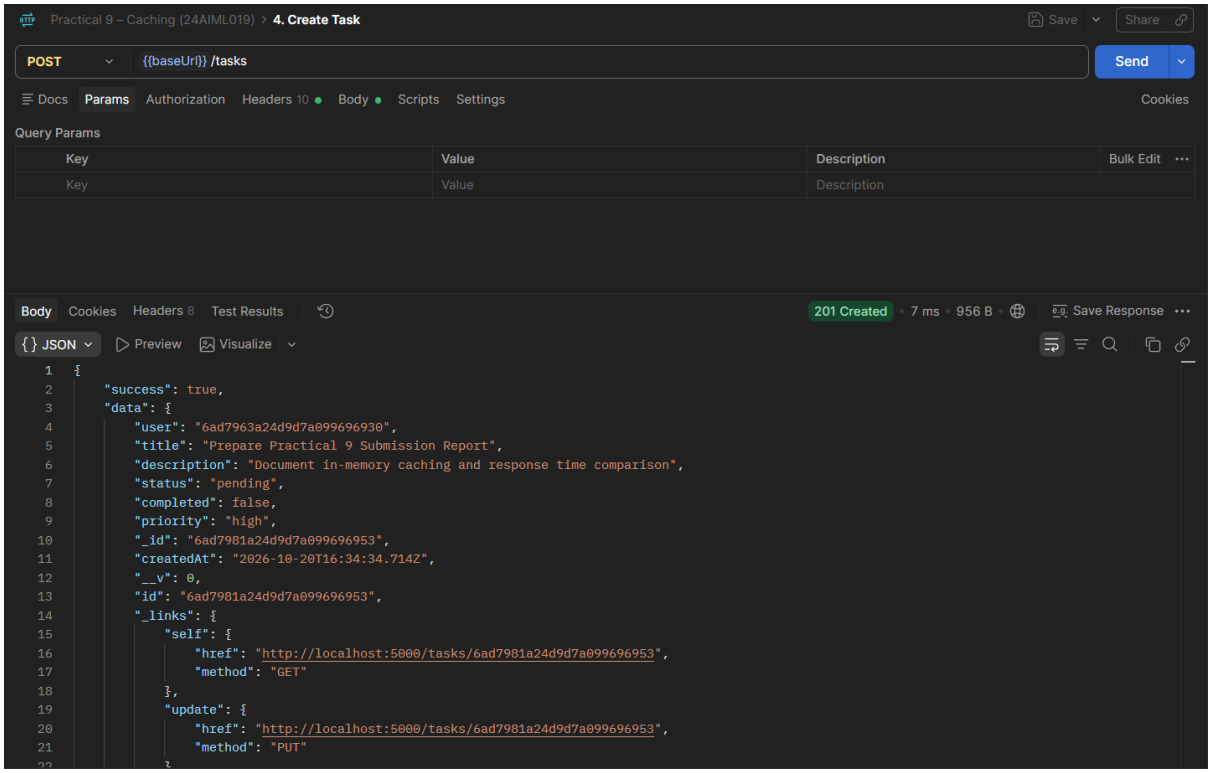


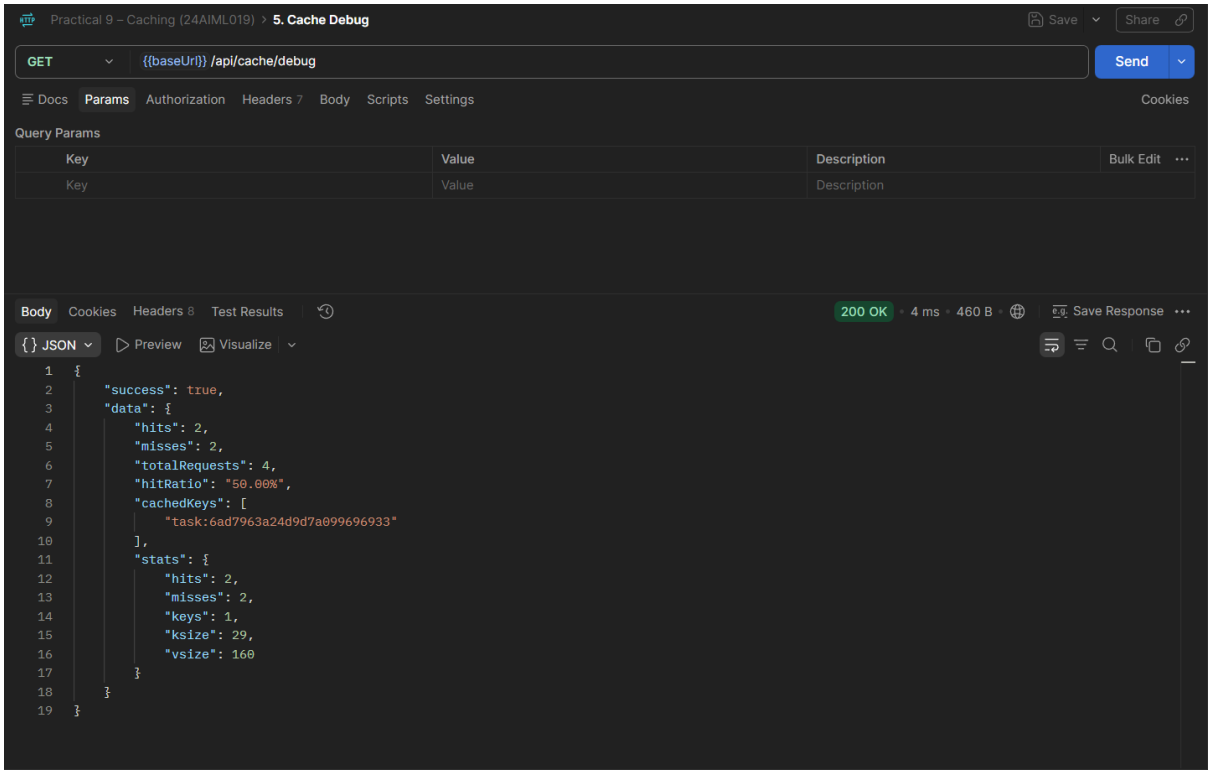
24AIML019

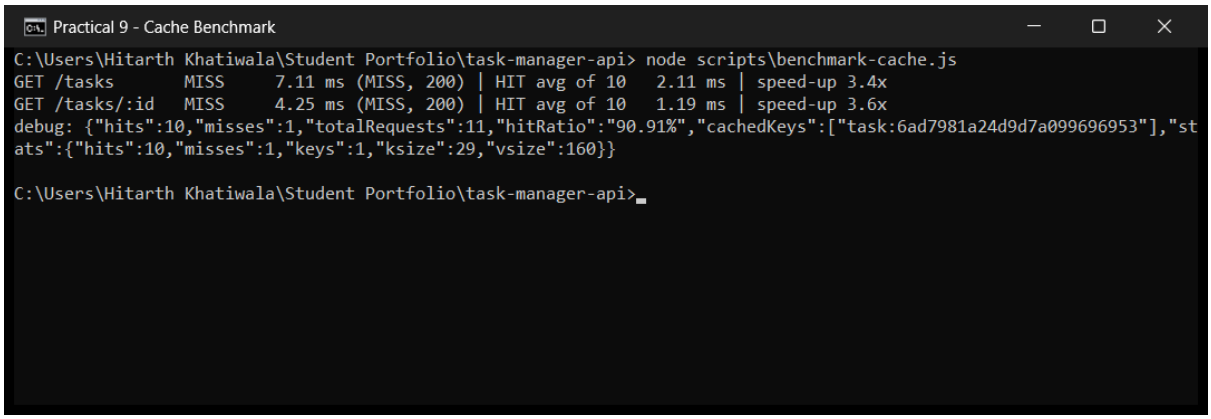

Practical 9 – Caching (24AIML019) > 2. Get All Tasks
Save
Share
GET
{{baseUrl}} /tasks
Send
Docs
Params
Authorization
Headers 7
Body
Scripts
Settings
Cookies
Query Params
Key
Value
Description
Bulk Edit
Key
Value
Description
Body
Cookies
Headers 10
Test Results
200 OK
7 ms
900 B
Save Response
Key
Value
X-Powered-By
Express
Access-Control-Allow-Origin
*
X-Cache
HIT
X-Cache-Lookup
in-memory (node-cache)
Content-Type
application/json; charset=utf-8
Content-Length
577
Etag
W/"241-6oSE6gtgLUirAgG81vCDolUGJVk"
Date
Tue, 20 Oct 2026 16:33:51 GMT
Connection
keep-alive
Keep-Alive
timeout=5


Practical 9 – Caching (24AIML019) > 3. Get Task By ID
Save
Share
GET
{{baseUrl}} /tasks/ {{taskId}}
Send
Docs
Params
Authorization
Headers 7
Body
Scripts
Settings
Cookies
Query Params
Key
Value
Description
Bulk Edit
Key
Value
Description
Body
Cookies
Headers 10
Test Results
200 OK
2 ms
1.09 KB
Save Response
{ } JSON
Preview
Visualize
"title": "Implement node-cache store",
"description": "Cache GET /tasks and GET /tasks/:id in memory",
"status": "pending",
"completed": false,
"priority": "medium",
"createdAt": "2026-10-20T16:26:34.193Z",
"__v": 0,
"id": "6ad7963a24d9d7a099696933",
"_links": {
"self": {
"href": "http://localhost:5000/tasks/6ad7963a24d9d7a099696933",
"method": "GET"
},
"update": {
"href": "http://localhost:5000/tasks/6ad7963a24d9d7a099696933",
"method": "PUT"
},
"delete": {
"href": "http://localhost:5000/tasks/6ad7963a24d9d7a099696933",
"method": "DELETE"
},

24AIML019


Practical 9 – Caching (24AIML019) › 4. Create Task
Save
Share
POST
{{baseUrl}} /tasks
Send
Docs
Params
Authorization
Headers 10
Body
Scripts
Settings
Cookies
Query Params
Key
Value
Description
Bulk Edit
Body
Cookies
Headers 8
Test Results
201 Created
7 ms
956 B
Save Response
{ } JSON
Preview
Visualize
{
"success": true,
"data": {
"user": "6ad7963a24d9d7a099696930",
"title": "Prepare Practical 9 Submission Report",
"description": "Document in-memory caching and response time comparison",
"status": "pending",
"completed": false,
"priority": "high",
"_id": "6ad7981a24d9d7a099696953",
"createdAt": "2026-10-20T16:34:34.714Z",
"__v": 0,
"id": "6ad7981a24d9d7a099696953",
"_links": {
"self": {
"href": "http://localhost:5000/tasks/6ad7981a24d9d7a099696953",
"method": "GET"
},
"update": {
"href": "http://localhost:5000/tasks/6ad7981a24d9d7a099696953",
"method": "PUT"


Practical 9 – Caching (24AIML019) › 5. Cache Debug
Save
Share
GET
{{baseUrl}} /api/cache/debug
Send
Docs
Params
Authorization
Headers 7
Body
Scripts
Settings
Cookies
Query Params
Key
Value
Description
Bulk Edit
Body
Cookies
Headers 8
Test Results
200 OK
4 ms
460 B
Save Response
{ } JSON
Preview
Visualize
{
"success": true,
"data": {
"hits": 2,
"misses": 2,
"totalRequests": 4,
"hitRatio": "50.00%",
"cachedKeys": [
"task:6ad7963a24d9d7a099696933"
],
"stats": {
"hits": 2,
"misses": 2,
"keys": 1,
"ksize": 29,
"vsize": 160
}
}
}



Github Repo Link: https://github.com/hitarth1812/Student-Portfolio

Key Questions:

• Why must the cache be invalidated on every write operation, and what would happen to data correctness if it were not?
Ans: Whenever a task is created, updated or deleted, the data in MongoDB changes and any cached copy of it becomes outdated, so it has to be removed; only then does the next GET request read the current data from the database. If the cache were left untouched, clients could receive outdated **or** inaccurate task details, such as a task that was already deleted, and the API would no longer match the database.

• What is a reasonable TTL (time-to-live) for cached data in a task management context, and what trade-off does TTL length represent?
Ans: A TTL between 60 seconds and a few minutes suits task data, and this lab uses 60 seconds. A short TTL means users see newer data, at the cost of more MongoDB queries; a long TTL makes the API faster and lowers database load, but cached data can stay out of date for longer. Choosing a TTL is therefore a compromise between accuracy and speed.

• Why is in-memory caching (node-cache) not suitable for a multi-server/multi-instance deployment, even though it works fine in this lab?
Ans: node-cache lives inside the memory of one Node.js process, so it cannot be shared. When the API is scaled to several servers, every instance holds its own copy of the cache, a write on one server leaves stale entries on the others, and all cached data disappears when a process restarts. For this reason, multi-instance setups use a central cache such as **Redis**, which all servers share and clear in one place.